COVID-19 Government Measures and Progression of the Pandemic

Berkay Özkırlı
boezkirl@smail.uni-koeln.de

Abstract

Governmental measures implemented to curb the COVID-19 pandemic have been a subject of significant media discourse, and their impact on the progression of the virus remains a topic of public debate. In this study, I develop a machine learning pipeline, informed by a curated subset of government interventions, to address the **binary classification** task of predicting whether a country will experience a significant surge in COVID-19 cases within a 14-day horizon.

1 Problem Formulation and Dataset Analysis

My main goal is to predict whether a country will experience a surge in COVID-19 cases over the next two weeks. I treat this as a binary classification problem. For any country on day t, let C_t be the 7-day smoothed daily confirmed cases per million (`new_cases_smoothed_per_million`). I define the target label $y_t \in \{0, 1\}$ based on whether cases grow by at least 25% over the next 14 days:

$$y_t = \mathbb{I}\left(C_{t+14} \geq 1.25 \times C_t\right) \tag{1}$$

where $\mathbb{I}(\cdot)$ is 1 if the condition is true and 0 if it is false. I dropped the final 14 days of each country's time series because they do not have a valid future target.

Population Representation vs. Generalization: The dataset represents the population of nations affected by the SARS-CoV-2 pandemic during 2020–2021. However, the true population I aim to generalize to is *future global populations encountering novel, airborne respiratory pathogens*. While COVID-19 serves as the proxy, the underlying goal is to produce an early-warning system for future pandemics before vaccines are developed.

1.1 Data Integration and Preprocessing Pipeline

I combined COVID-19 data from Our World in Data (OWID) [4] with government intervention logs from ACAPS [3]. To build a clean dataset, I applied three main preprocessing rules:

1. **Filtering by Testing Data:** Early pandemic data is noisy due to a lack of testing. I only kept countries that reported smoothed daily testing data (`new_tests_smoothed_per_thousand`) at least 80% of the time. This ensures that a predicted surge reflects real viral spread rather than a sudden jump in testing capacity. Only 56 countries satisfied the requirement.
2. **Policy Feature Engineering:** Raw ACAPS logs only record the exact dates when policies (like curfews or lockdowns) started. If I used these raw dates directly, the feature columns would be mostly zeros. To capture the ongoing effect of these policies, I created two continuous features: `days_since_last_harsh_measure` (days elapsed since the last restriction) and `active_harsh_measures_90d` (total restrictions enacted over the last 90 days).
3. **Chronological Data Splitting:** Because time-series data depends heavily on the past, a random train-test split would leak future information into the training process. Instead, I

split the data chronologically using the global timeline (where Day 1 is the earliest record). I used Days 1–300 ($t \leq 300$) for model training and cross-validation, and Days 301–385 ($301 \leq t \leq 385$) as a held-out test set. For initial exploration, I further split the training epoch into Days 1–240 (train) and Days 241–300 (validation).

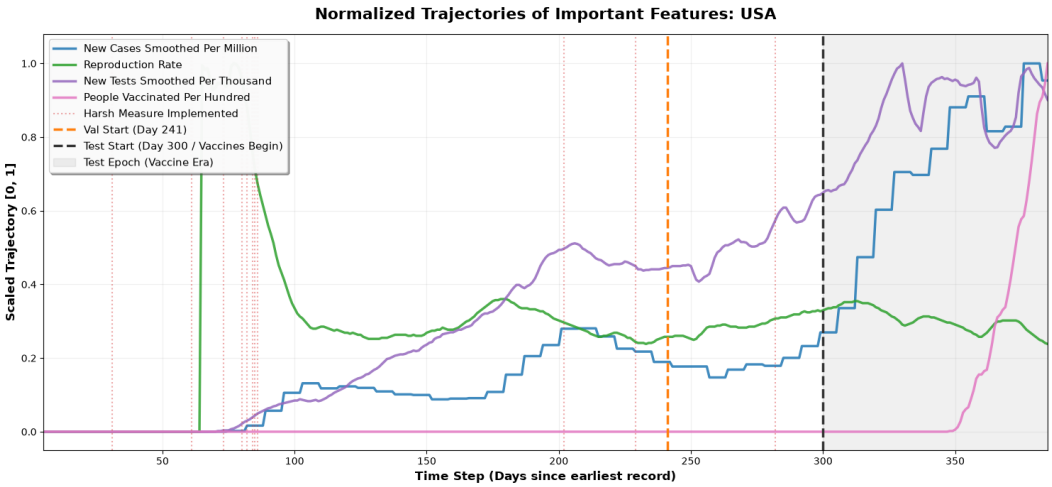


Figure 1: **Normalized feature trajectories for the United States (USA).** Min-Max scaling allows simultaneous visualization of testing capacity, viral reproduction momentum (R_t), and vaccination rollouts against government interventions (red dotted lines). The vertical dashed lines denote our chronological train ($t < 241$), validation ($241 \leq t \leq 300$), and test ($t > 300$) boundaries.

High-Level Statistics and Class Distribution: Following the testing coverage filter and target generation, the final machine learning matrix consisted of 56 countries, yielding 15,708 training/validation observations and 3,834 held-out test observations.

Before splitting, the overall target distribution was exceptionally well-balanced, with surges (Class 1) comprising 51.75% of the dataset and non-surges (Class 0) comprising 48.25%. However, examining the temporal splits reveals a distribution shift: the training set ($t \leq 240$) exhibited a high surge rate of 60.99%, reflecting the initial, unchecked waves of the pandemic. In contrast, the validation set ($241 \leq t \leq 300$) saw surges drop to 45.77%, and the test set ($t > 300$) plummeted to 30.57%. This decreasing baseline highlights the shifting dynamics of the pandemic (due to interventions and vaccines).

1.2 Data Leakage Audit and Feature Selection Rationale

Data leakage happens when a model uses information during training that would not be available in real life at prediction time t. I audited every feature to ensure it was strictly observable at day t, as summarized in Table 1.

I dropped retrospective metrics like excess mortality because they are calculated months later. I also dropped all vaccination metrics (`total_vaccinations`, `people_vaccinated`). Global vaccine rollouts did not start until after Day 360, meaning these columns are 100% missing values during my training epoch ($t \leq 300$), as shown by the flat zero-baseline in Figure 1. Including them would have confused the model during training and caused major errors during testing.

In addition to filtering epidemiological data, I filtered the ACAPS policy interventions. Out of 35 of available categories, I selected only five major restrictions: partial lockdowns, full lockdowns, isolation and quarantine policies, declared states of emergency, and curfews. I excluded minor or highly specific administrative measures (such as public awareness campaigns or localized guidelines) because they suffer from reporting noise and bureaucratic differences between governments. By focusing only on high-impact, universally recognized restrictions, I ensured that a policy trigger represents a comparable effect across all countries in my cohort.

Finally, I removed redundant demographic features to keep the model simple. As shown in Figure 2, age features are almost identical (`median_age` vs. `aged_65_older`: $r = 0.91$; `aged_65_older` vs. `aged_70_older`: $r = 0.99$). I kept `aged_65_older` as my single age baseline and dropped the rest.

1.3 Exploratory Data Analysis and Collinearity Diagnostics

To check how my selected features relate to each other and to the target variable, I plotted a Pearson correlation matrix (Figure 3). This revealed three main insights:

- **Reproduction Rate is the Strongest Signal:** The reproduction rate (R_t, labelled `reproduction_rate`) has the strongest positive correlation with `target_surge`. This confirms that current viral momentum is the best indicator of a future surge.
- **Policy Features Work as Expected:** My engineered policy features show logical correlations with viral growth. The variable `days_since_last_harsh_measure` correlates positively with surges, meaning cases are more likely to jump the longer a country goes without restrictions. Conversely, `active_harsh_measures_90d` correlates negatively, confirming that active restrictions help slow down the virus over time.
- **Low Redundancy:** Aside from obvious links between cumulative totals (like total cases and total deaths), the dynamic features do not heavily overlap. This confirms that my feature selection successfully removed unnecessary noise.

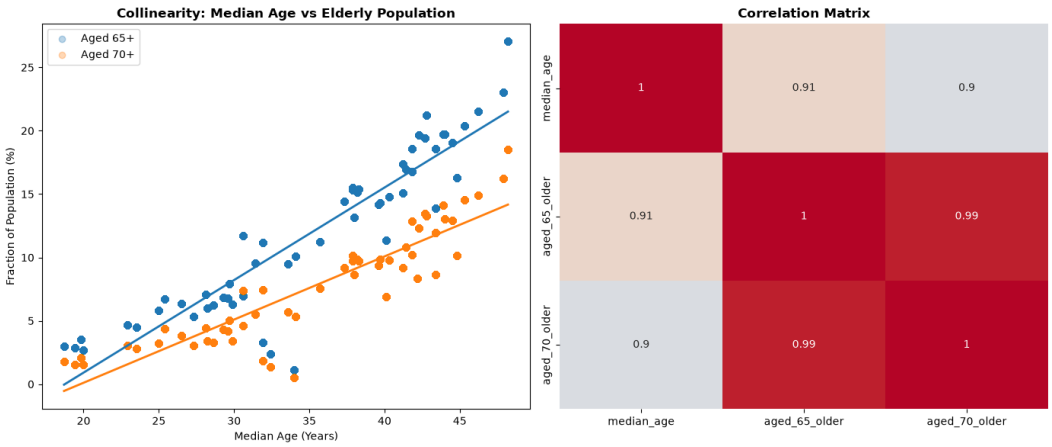


Figure 2: **Empirical justification for feature redundancy pruning.** The linear regression scatter plot (left) and Pearson correlation matrix (right) demonstrate extreme collinearity between median age and the elderly population fraction (`aged_65_older`: $r = 0.91$, `aged_70_older`: $r = 0.90$, and between the two elderly brackets: $r = 0.99$). To prevent tree-splitting bias and reduce dimensionality, only `aged_65_older` was retained as a representative demographic risk factor.

Table 1: **Data leakage audit and feature selection rationale.** Variables were evaluated for temporal availability, redundancy, and reporting consistency.

Variable	Available at t?	Included?	Reason
Smoothed case and death rates	Yes	Yes	Real-time viral momentum
Reproduction rate & stringency	Yes	Yes	Core transmission metrics
Demographics & beds per capita	Yes	Yes	Baseline population risk
Recent lockdown history	Yes	Yes	Causal policy context
14-day future cases (Target)	No	No	Direct data leakage
Excess mortality	No	No	Retrospectively calculated
Vaccination rates	No (in train epoch)	No	Unavailable during training
Median age	Yes	No	Redundant with age 65+
Raw daily cases	Yes	No	Noisy reporting artifacts

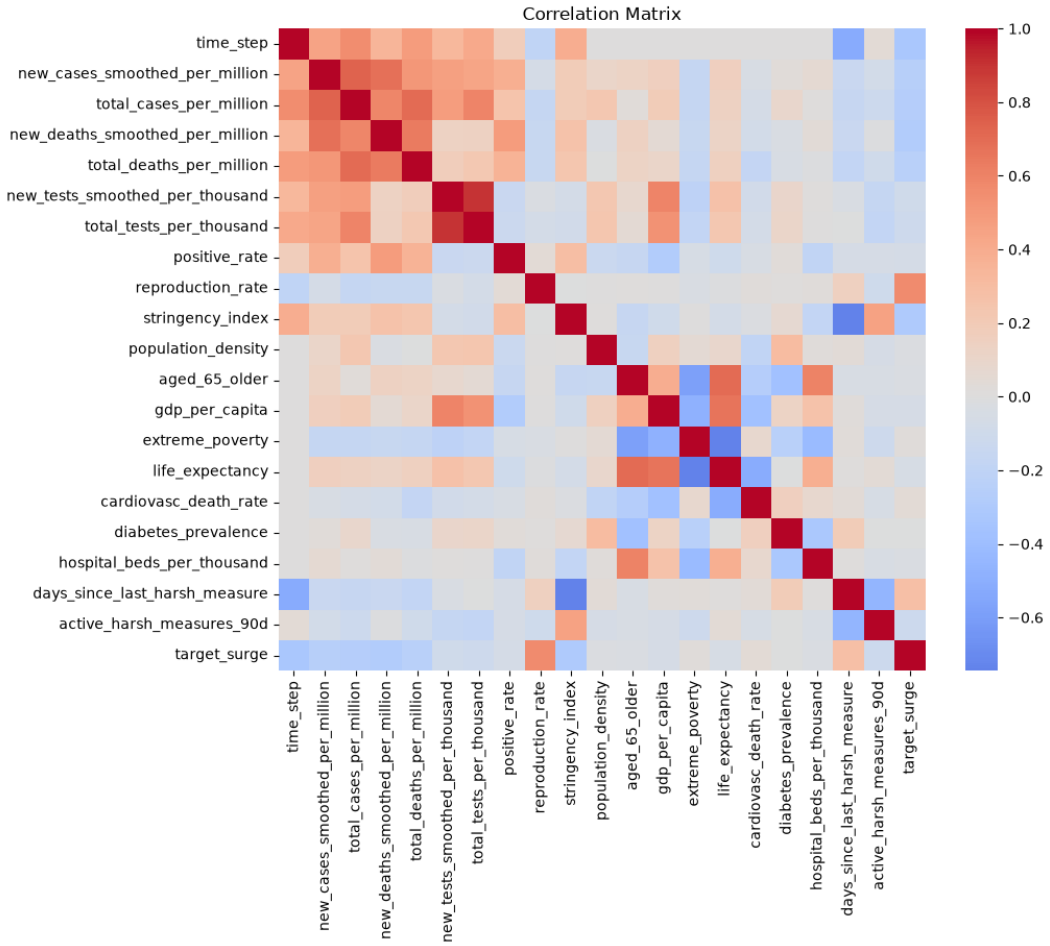


Figure 3: **Pearson correlation matrix across all retained machine learning features and the target variable (`target_surge`).** The viral reproduction rate (R_t) shows the strongest positive linear association with upcoming surges. Among policy indicators, `days_since_last_harsh_measure` correlates positively with surge occurrence, whereas our 90-day rolling intervention count (`active_harsh_measures_90d`) correlates negatively, confirming intiution behing the addition of the ACAPS data.

Fairness and Potential Harms: By explicitly filtering the cohort to include only countries with a $\geq 80\%$ reporting rate for daily testing volume, I inadvertently restricted the model to wealthier, developed nations with robust healthcare infrastructure. If this model were deployed globally as a policy early-warning system, it would likely perform systematically worse for developing nations or marginalized communities where testing is sparse or delayed. Consequently, relying on this model to allocate international emergency resources (e.g., WHO funding or ventilators) could disproportionately harm under-resourced regions by failing to detect their surges, as their sparse data would not match what the model learned from OECD nations.

2 Model Evaluation and Comparison

To evaluate how well different machine learning strategies predict viral surges, I compared three distinct model families against a trivial reference baseline:

1. **Simple Linear Model:** A standard Logistic Regression classifier to test if linear, additive relationships are sufficient.

2. **Tree-Based Ensemble:** An XGBoost Classifier [2] to capture non-linear decision boundaries and feature interactions without assuming a temporal sequence.
3. **Neural Network:** A Causal 1D Temporal Convolutional Network (1D-TCN) [1] utilizing dilated causal convolutions to explicitly learn temporal dependencies and historical context.
4. **Trivial Reference Baseline:** A naive "Persistence Predictor." In time-series forecasting, persistence assumes that future momentum matches current momentum. My persistence baseline predicts a future surge ($y_t = 1$) if the country experienced case growth over the preceding 14 days.

2.1 Pre-Experiment Hypothesis

Before running the experiments, I intuitively expected the **Causal 1D-TCN to perform best**. My reasoning was that COVID-19 transmission is inherently a continuous sequential process governed by lagging signals (e.g., hospitalizations lagging infections by weeks). I hypothesized that a deep learning architecture designed specifically for sequence modeling would capture these long-range temporal dependencies better than tabular models like XGBoost, which evaluate each time step as an independent row.

Table 2: **Comparison of three machine learning models and the persistence benchmark.** XGBoost outperforms the others, while Logistic Regression fails even against the naive benchmark.

Method	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC
Persistence	0.6699	0.6502	0.6032	0.6258	0.6647	0.5738
Logistic Regression	0.5667	0.5499	0.2935	0.3828	0.5893	0.5342
Causal 1D-TCN	0.7514	0.6990	0.8023	0.7471	0.8297	0.8309
XGBoost Classifier	**0.8342**	**0.8666**	**0.7537**	**0.8062**	**0.8868**	**0.8845**

2.2 Empirical Results and Analysis

I evaluated all models on the validation set using standard classification metrics. As shown in Table 2, the empirical results contradicted my initial hypothesis: **XGBoost significantly outperformed both the deep sequential model and the linear baseline.**

The Failure of Linear Modeling: Logistic Regression performed poorly, achieving an F1-Score of just 0.3828 and failing to beat even the simple persistence baseline (F1: 0.6258). This happens because linear models assume additive, independent relationships. However, viral spread is governed by multiplicative growth and threshold effects (e.g., transmission explodes only when $R_t > 1.0$). Forcing linear decision boundaries onto highly non-linear biological data results in severe underfitting.

2.3 Why Did Tree Boosting Outperform Deep Sequence Modeling?

The Causal 1D-TCN performed well (F1: 0.7551, ROC-AUC: 0.8346), proving that it successfully learned temporal patterns, as demonstrated by the stable convergence of its training loss curve in Figure 4. However, XGBoost achieved a superior F1-Score of 0.8062. I attribute this observation to two main factors in the dataset:

1. **Usefulness of Thresholds:** Tree-based models excel at finding sharp, non-linear rules. As shown in my feature importance analysis (Table 3), XGBoost relied heavily on exact numerical thresholds in `total_cases_per_million` (59.9% total gain) and `reproduction_rate` (21.9% gain). Decision trees capture sudden state changes like going over a reproduction threshold instantly without needing a trailing historical window.
2. **Tabular Feature Interactions:** XGBoost easily combines disparate feature types—such as multiplying real-time viral growth (R_t) by static healthcare capacity (`hospital_beds_per_thousand`) and recent policy stringency (`active_harsh_measures_90d`). While deep sequence models excel at pure audio or text sequences, gradient-boosted trees remain the most effective tool for structured tabular datasets with mixed continuous and static data types.

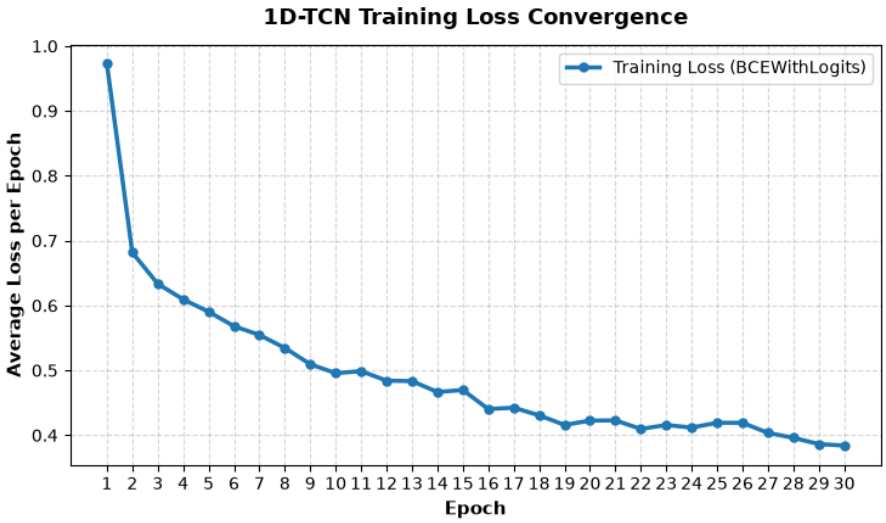


Figure 4: **Training loss curve for the Causal 1D-TCN.** The steady convergence confirms that the sequential network successfully learned temporal representations without severe overfitting, though it remained slightly less competitive than tabular tree boosting on noisy epidemiological data.

Table 3: **Top features by XGBoost Information Gain.** The model relies primarily on current viral prevalence and transmission momentum, with my engineered policy feature providing essential secondary context.

Feature Name	**Epidemiological Interpretation**	**Relative Gain (%)**
`total_cases_per_million`	Baseline cumulative viral prevalence	59.92%
`reproduction_rate` (R_t)	Instantaneous transmission velocity	21.94%
`new_deaths_smoothed`	Lagging outbreak severity indicator	7.28%
`new_cases_smoothed`	Immediate 7-day infection volume	4.78%
`active_harsh_measures_90d`	**Engineered:** 90-day restriction count	**2.28%**
`life_expectancy`	Static baseline demographic risk	1.91%
`hospital_beds_per_thousand`	Static healthcare capacity baseline	1.89%

3 Final Pipeline, Cross-Validation, and Ablation Study

3.1 Pipeline Architecture and Leakage Prevention

To ensure complete isolation between training knowledge and evaluation metrics, I deployed the final model via a scikit-learn `Pipeline`. This pipeline encapsulates feature scaling (`StandardScaler`) and classification (`XGBClassifier`), ensuring that normalization means and variances are strictly computed on training folds and merely applied to validation and test sets without data leakage.

3.2 Cross-Validation Strategy and Hyperparameter Tuning

Due to the autoregressive nature of epidemiological data, standard random K-Fold cross-validation would cause severe target leakage by allowing models to train on future viral states. Therefore, I implemented a strict **Temporal Walk-Forward Split**:

- **Training and Validation Epoch:** Days 1 to 300 ($15,708$ observations).
- **Held-Out Test Epoch:** Days 301 to 385 ($3,834$ observations).

For model tuning within the training epoch, I utilized a 5-fold `TimeSeriesSplit` with an explicit `gap=14`. This 14-day buffer prevents the validation folds from peeking into the future overlap created by my 14-day prediction horizon target label (y_t).

I tuned the XGBoost classifier using `RandomizedSearchCV`. Rather than an exhaustive brute-force search, random search explores a wide hyperparameter space efficiently. I evaluated 40 candidate configurations across the 5 temporal folds (totalling 200 model fits), optimizing specifically for the **F1-Score** to account for shifting class distributions over time.

The optimal hyperparameters found were: `n_estimators=500`, `max_depth=4`, `learning_rate=0.01`, `subsample=0.6`, `colsample_bytree=0.8`, and `gamma=0.1`. This tuned configuration achieved a cross-validated F1-score of 0.8670 on the training epoch.

3.3 Final Test Set Performance and Error Analysis

When evaluated on the unseen test set (Days 301–385, $N = 3,834$), the final XGBoost pipeline achieved an **Accuracy of 83.85%**, an **ROC-AUC of 0.8990**, and an **F1-Score of 0.7057**. The overall error distribution was heavily weighted toward correct classifications, with True Negatives representing 64.50% of the test epoch, True Positives at 19.35%, False Negatives at 11.22%, and False Positives at just 4.93%.

To understand why the model failed on the remaining 16.15% of test cases, I investigated the error groups:

- **False Positives ($N = 189$, 4.93%):** These occurred primarily when a country experienced a prolonged period without restrictions (`days_since_last_harsh_measure` averaged 198.8 days compared to 94.8 in True Negatives). When the reproduction rate spiked ($R_t = 1.06$), the absence of recent policy interventions tricked the model into predicting an unhindered exponential surge, even though absolute case volume (172.7 cases/million) failed to cross the 25% growth threshold.
- **False Negatives ($N = 430$, 11.22%):** Conversely, missed surges occurred in environments with heavy, ongoing restrictions (`active_harsh_measures_90d` averaged 3.07 rules, higher than any other group). The model learned from training data that active lockdowns suppress transmission. However, during the late-2020 test epoch, public "lockdown fatigue" caused transmission to spike despite active policies.
- **Locations:** 2 out of the top 5 countries where misclassifications occured were isolated island territories (Guam with 31, Sri Lanka with 25).

3.4 Ablation Study: Evaluating Policy Feature Engineering

To satisfy the requirement of evaluating a meaningful feature-design choice, I conducted a controlled ablation study on the test set. I dropped the two engineered temporal policy variables (`days_since_last_harsh_measure` and `active_harsh_measures_90d`) from the dataset, leaving only 17 features. I then re-ran the exact same 200-fit randomized search and evaluated the ablated pipeline on the held-out test set. The comparative results are presented in Table 4.

Table 4: **Controlled ablation study on the held-out test set ($N = 3,834$).** Removing my engineered policy features results in a marginal drop in recall and F1-Score, but slightly higher precision, highlighting a classical precision-recall tradeoff.

Model Configuration	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC
Full Pipeline (With Policy Eng.)	0.8385	0.7970	**0.6331**	**0.7057**	0.8990	0.8185
Ablated Pipeline (No Policy Eng.)	**0.8388**	**0.8037**	0.6254	0.7035	**0.9007**	**0.8268**
Difference (Full − Ablated)	-0.0003	-0.0067	$+0.0077$	$+0.0022$	-0.0018	-0.0082

Discussion of the Ablation Paradox: The ablation study revealed that removing the policy features caused almost no loss in overall accuracy (-0.03%) and slightly improved PR-AUC ($+0.82\%$). However, inspecting the individual classification metrics explains why I chose to **keep the engineered features in the final pipeline**:

1. **Precision vs. Recall Tradeoff:** Including policy history improves model **Recall by** $+0.77\%$ and net **F1-Score by** $+0.22\%$. In an early-warning system, missing a surge (False Negative)

is far more dangerous than issuing a false alarm. The policy features help the model detect an additional $+0.77\%$ of real outbreaks that the ablated model missed.

2. **Why Did Precision Drop Slightly?** As observed in my error analysis, when a country enacts a harsh measure, the full model expects case growth to slow down. If the public suffers from "lockdown fatigue" and ignores the rules, cases continue to rise anyway, causing the full model to misclassify the event. Without policy data, the ablated model relies purely on viral momentum (R_t and case counts), making it slightly more precise during periods of policy failure, but less capable of anticipating policy-driven recoveries.

Ultimately, keeping the engineered policy features is scientifically justified: they improve sensitivity to imminent surges, and ranked consistently among the top 10 most informative features by XGBoost gain (Table 3).

Collaboration and Citation Policy

Official documentation for torch, xgboost and scikit-learn were utilized. AI was used extensively in generating pandas data manipulations such as joins and the dataloader for the deep learning model. AI was also used in general brainstorming. Gemini was the model of choice.

References

[1] Bai, S., Kolter, J. Z., & Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*.

[2] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD*.

[3] ACAPS. (2020). COVID-19 Government Measures Dataset. *Assessment Capacities Project*.

[4] Mathieu, E., et al. (2020). Coronavirus Pandemic (COVID-19). *Our World in Data*.